

第 8 讲端到端策略导论：从 Pipeline 到 Policy

目录

1 两种机器人：手写 pipeline 还是端到端 policy	2
1.1 传统机器人怎么干活：一条手写的流水线	2
1.2 流水线的死穴：有些环节根本写不出规则	2
1.3 换个思路：让一个网络从观测直接到动作	3
2 什么是 policy	4
2.1 一句话定义：从观测到动作的映射	4
2.2 观测里有什么、动作又是什么	4
2.3 策略的谱系：从只看状态到看懂语言的 VLA	5
2.4 本节小结	6
3 从训练到部署：为什么 loss 低了还会失败	6
3.1 训练损失低不等于任务成功	7
3.2 闭环里误差会滚雪球：为什么必须 rollout 验证	7
3.3 动作口径：绝对还是相对，两端必须说同一种话	8
3.4 归一化：把统计量和模型绑在一起	10
3.5 裁剪、限位与出发前检查单	12
3.6 本节小结	13
4 实验：用 π_0 跑通第一个策略闭环	13
4.1 先认识主角： π_0 当一个黑盒策略	13
4.2 搭好舞台：LIBERO 仿真环境	14
4.3 闭环跑起来：看一眼、动一下	14
4.4 拿着检查单核对这次部署	14
5 本讲小结	15
5.1 回顾整条数据流	15
5.2 策略要吃数据：下一讲采数据	15
参考资料	15

1 两种机器人：手写 pipeline 还是端到端 policy

让一台机械臂把桌上的方块放进盘子，有两种截然不同的做法。一种是把”怎么动”这件事一步步算出来，每一环都人写规则；另一种是直接用一个网络从相机画面映射到电机指令。这一讲从这两种机器人的对比讲起，引出后面六讲的共同主角——策略 (policy)。

1.1 传统机器人怎么干活：一条手写的流水线

传统做法会把”怎么动”**显式地**用人写的模型一步步算出来：先感知与建图，再用 RRT、MPC 这类算法规划一条轨迹，最后交给控制器执行——每一环都是人设计的算法。

把”生成机器人运动”的方法摊开看，大致分成两支：左边是这种**显式建模**（运动学、规划、控制），右边是这一讲要讲的**隐式建模**（从示范中学习、强化学习）：

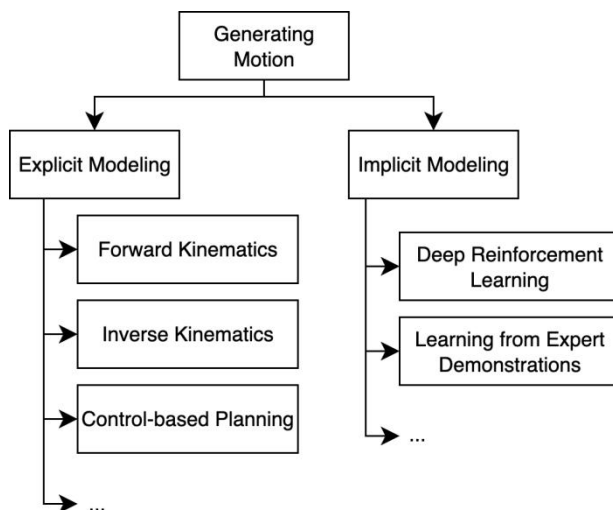


图 1: 生成机器人运动的两条路：左侧显式建模（运动学、规划、控制），右侧隐式建模（模仿学习、强化学习）

显式流水线的好处是每一步都看得见、查得到——哪里出错就去修哪一段，可解释、可调试。

1.2 流水线的死穴：有些环节根本写不出规则

问题在于，有些环节是写不出规则的。

抓一块软布、把线穿进针眼、拧一个滑手的瓶盖——这些任务里物体会形变、会打滑，接触力瞬息万变，你很难用解析模型把”此刻该怎么动”描述清楚。再加上真实世界的长尾层出不穷：换个光照、换个没见过的杯子，手写的感知和规划就要重新调一遍。

流水线不是不好，而是它把”难以言说”的技能硬拆成可编程的步骤，恰恰在最难的那几环上卡住了。

1.3 换个思路：让一个网络从观测直接到动作

端到端策略给出的答案是：不再把任务拆成感知—规划—控制三段分别设计，而是用一个神经网络，直接从原始观测映射到动作。相机像素进去，电机指令出来，中间不再由人显式编写。

备注：端到端不是要取代一切。正逆运动学、安全限位、底层伺服这些有成熟解析解的部分仍然保留；端到端替代的，是那几个“写不出规则”的环节。

最早把这件事讲清楚的两个工作，一个在自动驾驶、一个在机械臂。NVIDIA 的 PilotNet 让摄像头画面直接进一个卷积网络、输出方向盘转角，中间不再有“车道线检测 → 路径规划 → 转向控制”的人工流水线：

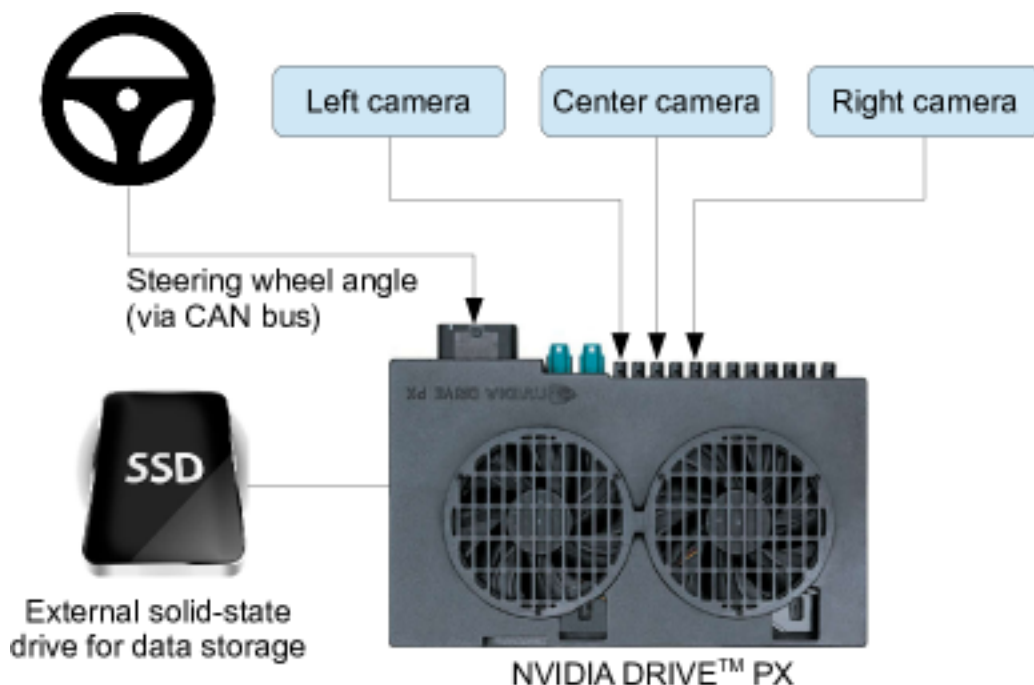


图 2: PilotNet：摄像头图像经一个卷积网络直接输出方向盘转角，端到端反向传播训练

Levine 等人在机械臂上做了同样的事，并把本讲的题眼问了出来——把感知和控制联合端到端地训，会不会比分开设计更好？他们用一个网络把图像直接映射到电机扭矩，答案是肯定的：

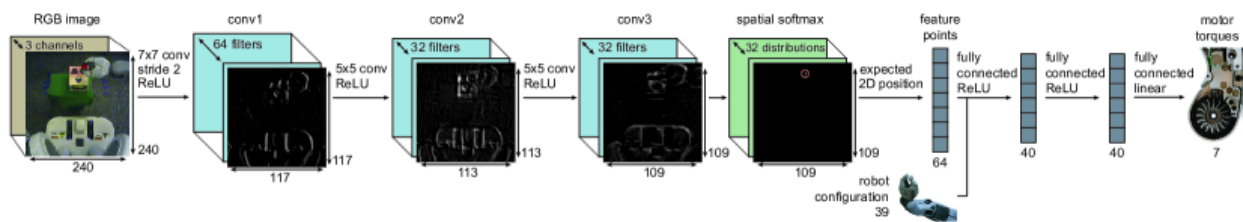


图 3: Levine 视觉运动策略：RGB 图像经卷积与空间特征点提取，直接映射到电机扭矩

把这两件事抽象出来——“那个直接吃观测、吐动作的网络”——就是这一讲真正的主角。

2 什么是 policy

2.1 一句话定义：从观测到动作的映射

策略（policy）就是一个从**观测**到**动作**的函数：你给它当前看到的东西，它告诉你下一步该怎么动。

放进机器人和环境的互动里看最清楚：智能体（机器人）从环境收到观测，输出一个动作作用回环境，环境再给出新的观测——如此往复。这张经典的”智能体—环境”回路就是策略工作的舞台：

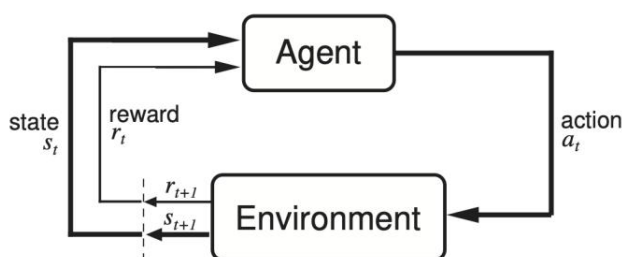


图 4: 智能体与环境的交互回路：智能体根据观测输出动作，环境接收动作后返回新的观测，循环往复

图里用了强化学习的习惯记号（状态 s_t 、动作 a_t 、奖励 r_t ），本讲先只关注”观测进、动作出”这一条主线，奖励要到讲 14 的强化学习才登场。写成符号是 $a = \pi(o)$ ；更一般地写成一个分布 $\pi(a | o)$ ——同一个观测下，合理的动作往往不止一个，这件事到讲 10 学生成式策略时会成为核心出发点。

符号	含义	典型例子
o	observation, 观测	相机图像、机器人关节角
a	action, 动作	各关节目标角度、末端位姿、电机扭矩
π	policy, 策略（要学的网络）	一个卷积 + 全连接网络

2.2 观测里有什么、动作又是什么

观测和动作具体长什么样？看一段真实遥操作数据最直观：每一帧的**观测** o_t 由机器人本体状态（关节角 q_t ，那串数字）加上相机图像组成，对应的**动作** a_t 是人类专家此刻通过主臂给出的目标关节角——策略要学的，就是”给定上面的观测、回归出下面的动作”：

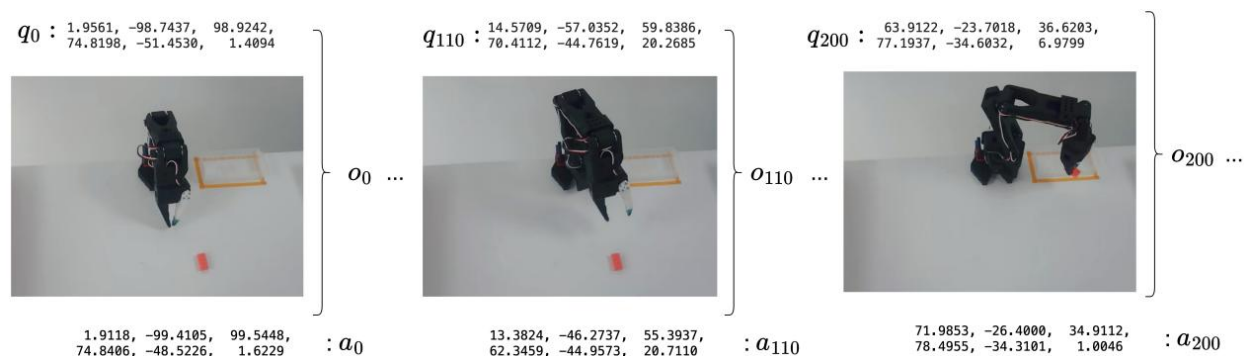


图 5: 一段遥控操作轨迹里的观测—动作配对: 每帧由本体状态加相机图像构成观测, 对应人类专家给出的目标关节角作为动作

由此能看清观测和动作各有哪些常见形态:

观测 o 可以是: 本体状态 (关节角、末端位置, 维度低、信息直接)、图像 (信息丰富但要网络自己提取)、语言指令 (让同一个策略能做不同任务), 以及它们的组合与一小段历史 (不只看当前一帧)。

动作 a 的几个关键选择: 在关节空间还是末端空间表达; 给**绝对**目标还是**相对**增量; 一次只预测单步, 还是预测未来一小段动作块 (action chunk)。

备注: 观测和动作的”口径”是策略的接口约定。同一个任务换一种动作表示 (比如关节角换成末端位姿、绝对换成相对), 对网络来说就是另一个学习问题——后面几讲的很多设计差异, 以及第 3 节部署翻车的坑, 都出在这里。

2.3 策略的谱系: 从只看状态到看懂语言的 VLA

把”观测里加不加图像、加不加语言”当坐标, 策略就排成一条谱系: 只看本体状态的 **state policy** → 加图像的**视觉运动策略** (如 ACT、Diffusion Policy) → 再加语言指令的**语言条件策略** (如 RT-1) → 把视觉、语言、动作统一进一个大模型的 **VLA** (如 π_0)。越往右, 策略”看懂”的东西越多、能听的指令越泛。这条谱系正好是本课程”端到端策略”这一大块 (讲 8–13) 的地图, 最右端的 VLA 会从讲 11 起展开。

谱系最右端的 VLA 到底”统一”了什么? 一句话: 把”看懂图像”的视觉模型、“看懂文字”的语言模型、“控制行为”的动作模型合成一个——让机器人既能看、又能听, 再把这份理解直接变成动作:

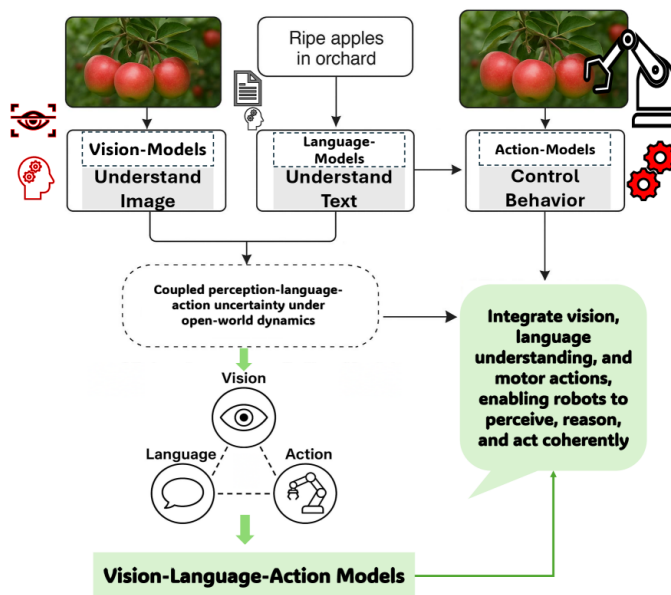


图 6: 视觉-语言-动作模型把视觉模型（看懂图像）、语言模型（看懂文字）、动作模型（控制行为）三者合一，让机器人能感知、推理并连贯地行动

2.4 本节小结

策略就是“观测 $\rightarrow$ 动作”的那个函数，写作 $\pi(a | o)$ 。它的两个接口——观测里放什么、动作怎么表达——决定了它是一个只看状态的 state policy，还是一个能听语言的 VLA，也决定了它好不好训、好不好部署。下一节就来看，当我们真把一个策略学出来、搬上机器人时，会在哪些地方栽跟头。

3 从训练到部署：为什么 loss 低了还会失败

策略是学出来的。但“学得好”和“用得好”之间，隔着几道部署时才会暴露的坎。这一节不讲新算法，讲**数据的变换**：示教数据在进网络之前，要先统一动作口径、再做归一化；部署时，网络吐出的数字要按**原路逆着走回来**——先反归一化、再还原口径，才变回机器人能执行的指令。这条“来回路”上任何一环两端对不上，机器人就会表现出三种经典症状：**几乎不动**、**越走越飘**（发散）、或者**来回抖动**——而此时网络和权重往往都没有任何毛病。

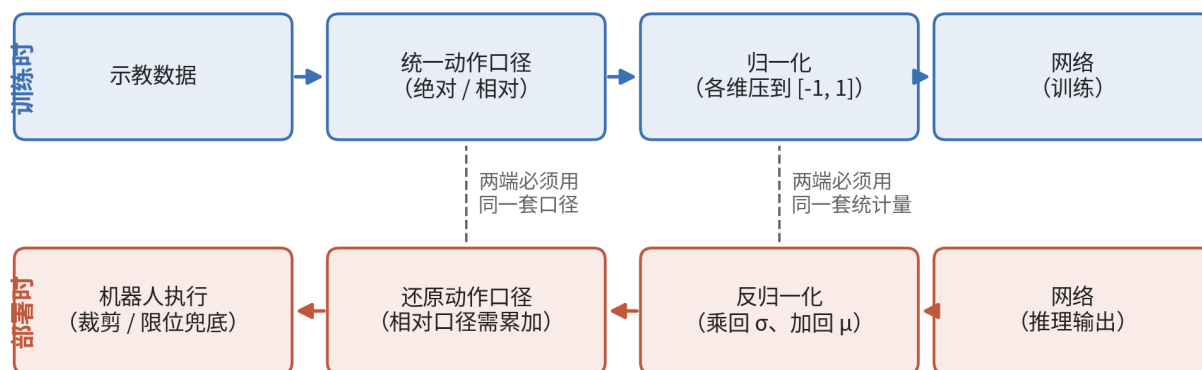


图 7: 训练时示教数据经过”统一动作口径、归一化”进入网络；部署时网络输出必须用同一套约定”反归一化、还原口径”逆着走回来，最后由裁剪限位兜底

上图就是本节的路线图：上排是训练时数据的去路，下排是部署时数据的回路，中间两条虚线是最容易断掉的两处衔接——动作口径（3.3）和归一化统计量（3.4）。在拆这两处衔接之前，先回答一个更基本的问题：怎么判断一个策略到底行不行（3.1、3.2）。

3.1 训练损失低不等于任务成功

最容易踩的认知误区，是盯着训练损失（loss）下降就以为大功告成。

策略的训练目标通常是”让预测动作接近示教动作”，这是一个**单步**的监督指标。但任务成功是一个**闭环**的结果：要一步接一步走到底、把方块真的放进盘子。单步每步差一点点，几十步累积下来可能完全跑偏；反过来，loss 不是最低的那个 checkpoint，真机成功率反而更高，也很常见。也不存在一个通用的数值，说”loss 降到多少就能部署”——loss 只能用来粗筛 checkpoint，最终的裁判永远是让策略真正跑一遍任务。

一句话：loss 衡量的是”像不像示教”，成功率衡量的是”任务成没成”，两者经常对不上——所以选模型不能只看 loss 曲线。

3.2 闭环里误差会滚雪球：为什么必须 rollout 验证

为什么单步差一点，闭环会差很多？因为机器人是用自己的动作决定自己下一步看到什么。

训练时策略见到的都是示教里”正确”的状态；一旦它在某一步偏离一点，就进入了训练时没见过的状态，于是下一步预测更偏、状态更陌生——误差像滚雪球一样越滚越大。下图画的就是这个过程：

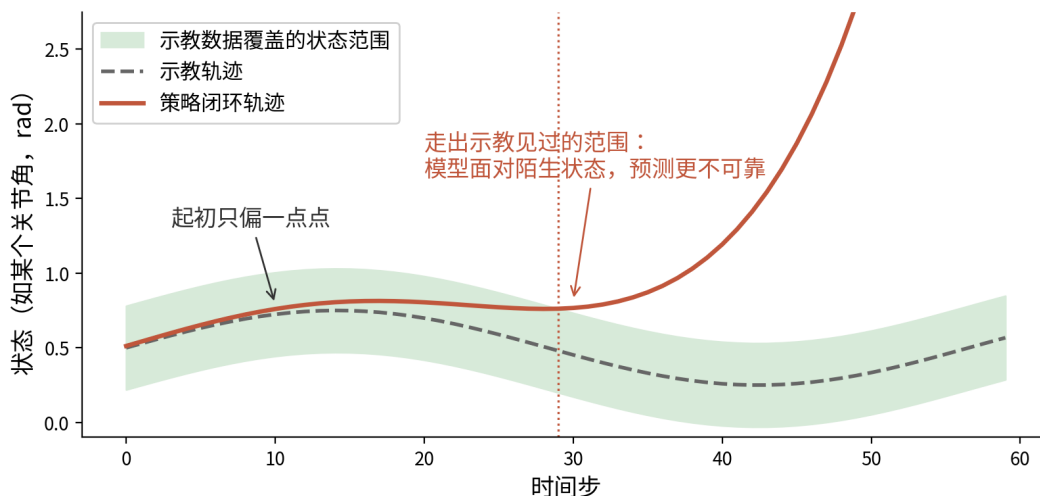


图 8: 绿色带是示教数据覆盖过的状态范围；策略的闭环轨迹起初只比示教偏一点点，一旦走出绿色带、进入模型没见过的状态，预测越来越不可靠，误差加速放大

注意图里的转折点：轨迹还在绿色带内时，偏差几乎看不出来；**走出带子的那一刻**，模型面对的是示教里从未出现过的状态，输出开始失去依据，轨迹加速飘走。这个现象叫**分布漂移** (covariate shift)，是模仿学习的经典难题，Ross 等人提出的 DAgger 就是冲它去的（机制完整展开留到讲 10）。

结论很硬：**评估一个策略，只能让它在环境里真正跑一遍 (rollout)、看任务成没成，不能只看离线指标**。这也是为什么本讲第 4 节的实验，核心不是算 loss，而是让策略在仿真里闭环跑起来。

3.3 动作口径：绝对还是相对，两端必须说同一种话

2.2 提过动作可以是**绝对**的，也可以是**相对**的（增量 / delta）。这是这一讲最该记牢的细节之一——它既影响网络好不好学，又是部署时最常翻车的地方。

先把两种口径说清楚。假设机械臂某个关节此刻在 1.18 弧度，下一帧示教把它转到了 1.20 弧度，那么“这一步动作”有两种记法：

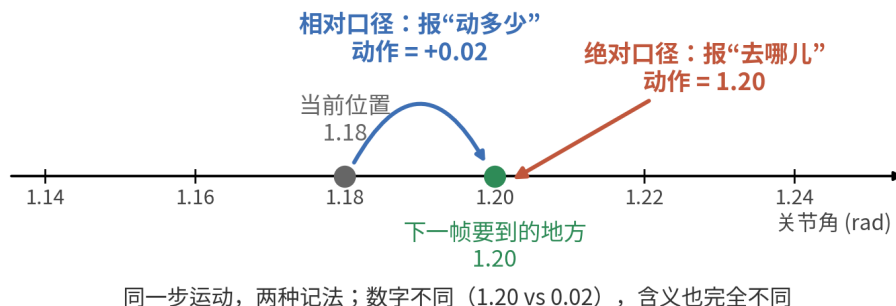


图 9: 同一步运动的两种记法：相对口径报“动多少” (+0.02)，绝对口径报“去哪儿” (1.20)；数字不同，含义也完全不同

- **绝对口径**：策略直接输出“目标 = 1.20 弧度”。动作的含义是**去哪儿**，跟你现在在哪没关系。
- **相对口径**：策略输出“在当前基础上 +0.02 弧度”。动作的含义是**动多少**，必须配合“当前在哪”才能还原出真正的目标 ($1.18 + 0.02 = 1.20$)。

末端位姿也是一样：可以给“末端在世界坐标系下的绝对位姿”，也可以给“末端再往前挪 1 厘米”这种增量。两种口径各有长短，没有绝对的好坏：绝对口径每一步都瞄准一个固定坐标，动作表示本身不会累积漂移，但网络得记住“整个工作空间长什么样”；相对口径是“局部修正”，跨起点泛化好、通常也更好学，代价是部署时要一步步把增量累加回绝对位置。口径的选择本身就影响性能——Diffusion Policy 的实验里，仅仅换一种动作表示（速度口径换成位置口径），多个任务的成功率就明显不同。

真正的分水岭在于：**训练和部署两端，必须用同一种口径去解释同一个数字**。注意相对口径有个数值特点——一个 30Hz 采集、动作温和的抓取任务，相邻两帧的关节增量常常只有 $0.01 \sim 0.03$ 弧度，几乎贴着 0。由此引出两个对称的坑，下图左右各画了一个：

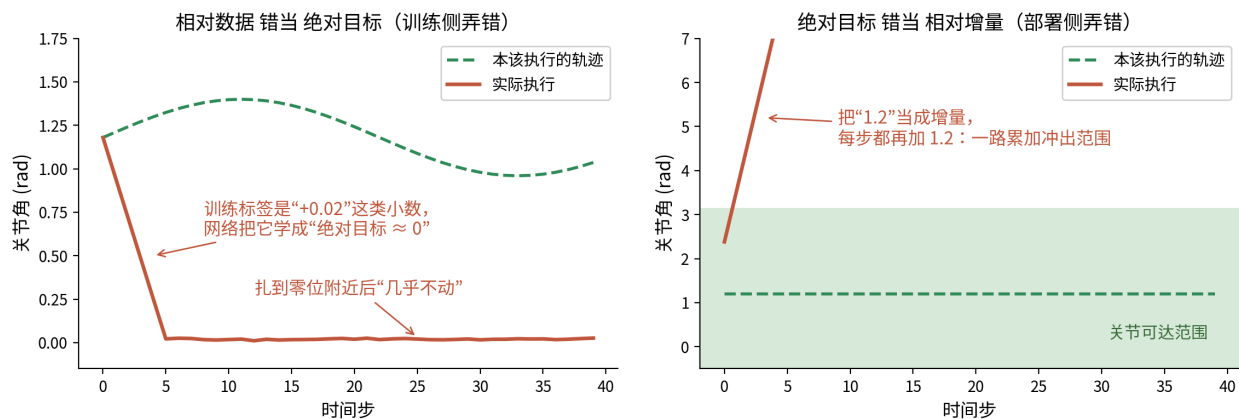


图 10: 左：相对数据被错当成绝对目标训练，网络学会输出“绝对目标约等于 0”，机械臂扎向零位后几乎不动；右：绝对目标被错当成相对增量部署，每步都在当前位置上再累加，关节角一路冲出可达范围

- **左图，训练侧弄错——把相对数据当绝对目标训**：本该是“+0.02”的增量，被错当成绝对目标喂给网络。网络于是学会“不管现在在哪，都输出一个 0.02 左右的绝对目标”——机械臂径直扎向

接近零位的姿态；到那之后，每步目标几乎不变、真实位移只有一个最小单位的量级，还会被预测噪声整个淹没。症状：**先猛地一动，然后几乎不动**。此时 loss 依然很漂亮、网络本身也没毛病，机器人却罢工了。

- **右图，部署侧弄错——把绝对目标当相对增量发**：训练时动作是绝对目标（比如 1.2），部署端却把它当成增量，每步都在当前位置上再加 1.2——关节角一路累加、冲出可达范围。症状：**单调地越走越远**（发散）。反过来，相对模型部署时忘了“目标 = 当前位置 + 增量”这步累加，把 0.02 直接当绝对目标下发，就回到左图的下场。

备注：动作口径必须在训练和部署两端**完全一致**——同一个数字 0.02，在绝对口径下意思是“去 0.02 弧度处”，在相对口径下意思是“再走 0.02 弧度”，差之毫厘谬以千里。这是一条约定层面的硬约束，不是事后调参能补救的。

备注：口径的约定其实从**采数据**那一刻就开始了。主从遥操作里从臂跟随主臂，也分两种方式——**绝对跟踪**：从臂直接去主臂当前的绝对姿态，录下的动作就是从臂真实到达的姿态，前提是主从标定的零点对齐；**相对（锚定）跟踪**：接通的第一帧记下“从臂当前姿态减主臂姿态”的差，之后从臂只复现主臂的位移。锚定跟踪不要求零点严格对齐、接通瞬间从臂不会突然跳动，但录下的动作和从臂实际姿态恒差一个**每次开机都不同的常数**——动作口径已经被悄悄改写，部署时更容易两端对不上。遥操作采数据的细节，下一讲展开。

3.4 归一化：把统计量和模型绑在一起

第二处容易断掉的衔接是**归一化**。

为什么非归一化不可。一份机器人数据里，不同物理量的数值范围天差地别：关节角可能在 $[-3.14, 3.14]$ 、夹爪开合是 $[0, 1]$ 、图像像素是 $[0, 255]$ 。神经网络对输入输出的数值尺度很敏感——若直接喂原始值，范围大的维度会主导损失、范围小的维度几乎学不动。所以训练前几乎都会把每一维**各自归一化**到相近的尺度：

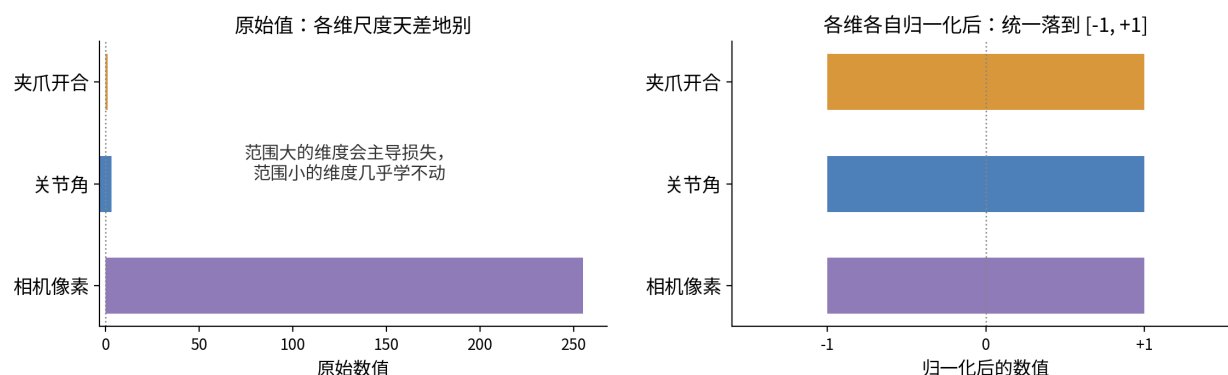


图 11: 左：相机像素、关节角、夹爪开合三种量的原始数值范围相差悬殊；右：各维各自归一化后，统一落到 $[-1, +1]$

常见两种做法。第一种是**均值-方差 (mean / std)**： $\hat{x} = (x - \mu) / \sigma$ ，用这一维的均值 μ 和标准差 σ 把数据拉成零均值、单位方差。简单，但对离群点敏感——数据里混进几个异常大的值（比如传感器毛刺）， σ 就被撑大，正常值反而被压扁。

第二种是**分位数 (q01 / q99)**。先说清 q99 是什么：把这一维的所有数据从小到大排队，排在 99% 位置上的那个数就是 q99——也就是说，99% 的数据都不超过它；q01 同理，是排在 1% 位置上的数。归一化时不用最小值和最大值，而用 $[q_{01}, q_{99}]$ 这段区间做线性缩放，把它映射到 $[-1, +1]$ ；落在区间外的、最极端的那约 1% 数据，直接截断到边界上：

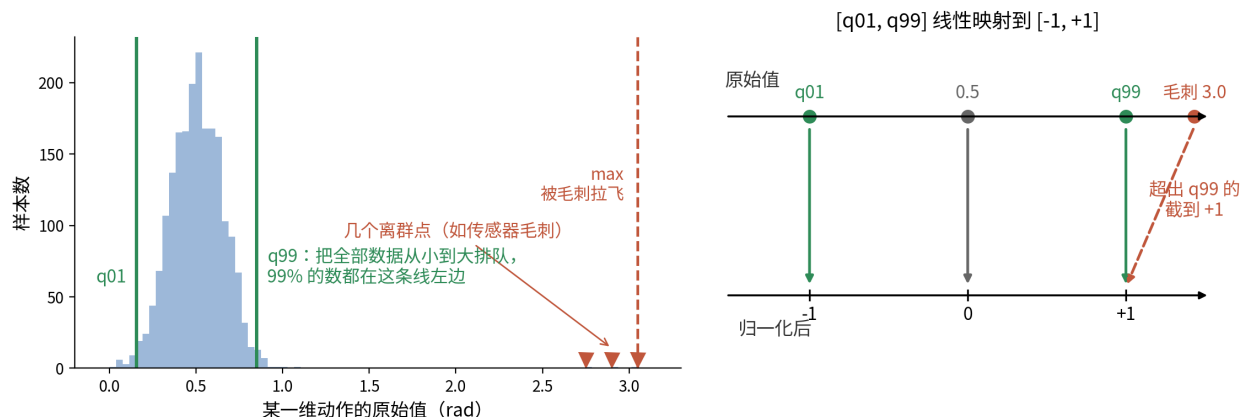


图 12: 左：直方图里几个离群点把 max 拉得很远，而 q01/q99 紧贴数据主体；右：把 $[q_{01}, q_{99}]$ 线性映射到 $[-1, +1]$ ，超出 q99 的毛刺被截断到 +1

好处在左图一眼可见：min / max 会被那几个毛刺整个带偏，q01 / q99 则稳稳贴住数据主体。所以分位数归一化更抗离群点—— π_0 、LeRobot 这类框架默认用的就是它。

举个具体的数走一遍归一化的“来回”。某关节在训练集上统计出 $\mu = 0.3$ 、 $\sigma = 0.5$ 。示教里一个目标 0.8，归一化后是 $(0.8 - 0.3) / 0.5 = 1.0$ ，网络学到的、吐出来的都是这个 1.0。轮到推理时网络输出 1.0，必须再**反归一化**回真实尺度： $1.0 \times 0.5 + 0.3 = 0.8$ ，这个 0.8 才是真正下发给机器人的值。训练（正向归一化）和部署（反向还原）用的，必须是同一套 μ, σ 。

关键在于：这套统计量 (μ, σ 或 q01 / q99) 是从训练集统计出来的，是模型的一部分——它通常和权重放在一起，存成一个不起眼的小文件。部署时如果只带走了权重、把统计量落下了（或者拿错成另一份数据的统计量），反归一化就退化**成“什么都不做”：网络输出的 1.0 被原样当成 1.0 发出去，而它本该被还原成 0.8。整条动作的尺度系统性地错位，症状是下图这两种：**

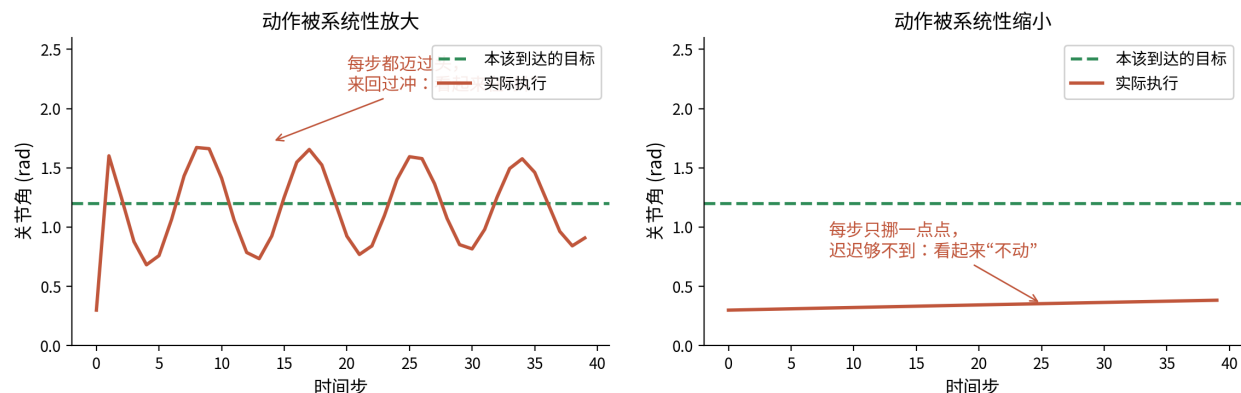


图 13: 左：动作被系统性放大，每步都迈过头、围绕目标来回过冲，看起来在“抖”；右：动作被系统性缩小，每步只挪一点点、迟迟够不到目标，看起来“不动”

尺度被放大，机器人每一步都迈过头、围着目标来回过冲——看起来在抖；尺度被缩小，每一步只挪一点点、迟迟够不到目标——看起来不动。这种错和 3.3 的坑同一个性质：网络、权重都没问题，全错在“模型和它的附属统计量分了家”。

一句话：模型权重和它的归一化统计量是一套东西；权重对了、统计量丢了或拿错了，输出照样全错。而且观测侧（输入）和动作侧（输出）两头都要归一化，部署时两套统计量一个都不能少。

3.5 裁剪、限位与出发前检查单

就算前面都对，真机上还要留一道兜底闸：把动作**裁剪**（clip）到合理范围、限制最大速度、加上关节限位。策略偶尔吐出一个离谱的值时，这道闸保证机器人不会突然甩出去——这是真机不出事的工程底线。

最后，把本节的内容收成一张**出发前检查单**，部署前逐行过一遍：

检查项	要确认的问题
动作口径	训练数据的动作是绝对还是相对？部署端按同一种口径解释了吗（相对口径记得累加还原）？
归一化统计量	统计量文件和权重在一起吗？观测侧、动作侧两套都齐吗？
数值量级	离线喂几帧观测，输出的数值量级像正常动作吗（相对口径应是贴近 0 的小数）？
起始位姿	机器人从示教数据里出现过的起始位姿附近出发了吗？
安全兜底	裁剪、限速、关节限位这道闸设了吗？
评估标准	判好坏的标准定成 rollout 成功率了吗（而不是 loss）？

这张表不长，但每一行背后都是一类“模型没毛病、机器人不工作”的故障。第 4 节的实验会拿着它逐

行核对一遍。

3.6 本节小结

把一个策略从训练搬到部署，本质是让数据把那条”来回路”走完整：训练侧怎么把示教数据变换进网络（统一动作口径、归一化），部署侧就必须用同一套约定逆着变换回来（反归一化、还原口径），最后用裁剪与限位兜底；判断成没成，只认闭环 rollout (3.1、3.2)。这些都不是算法创新，却是机器人能不能动起来的分水岭——“模型看着没问题、就是不工作”的故障，十有八九断在来回路的某一环上，症状无非三种：不动、发散、抖动。

4 实验：用 π_0 跑通第一个策略闭环

讲了这么多概念，这一节亲手跑一个真正的策略，把”看一眼、动一下”的闭环看在眼里。我们用一个现成的、能力很强的 VLA—— π_0 ——当作黑盒策略，在仿真环境 LIBERO 里加载它的预训练权重，跑一遍 rollout。

4.1 先认识主角： π_0 当一个黑盒策略

π_0 是 Physical Intelligence 提出的视觉-语言-动作模型：一个视觉-语言骨干网络看懂图像和指令，后面接一个”动作专家”生成连续动作。

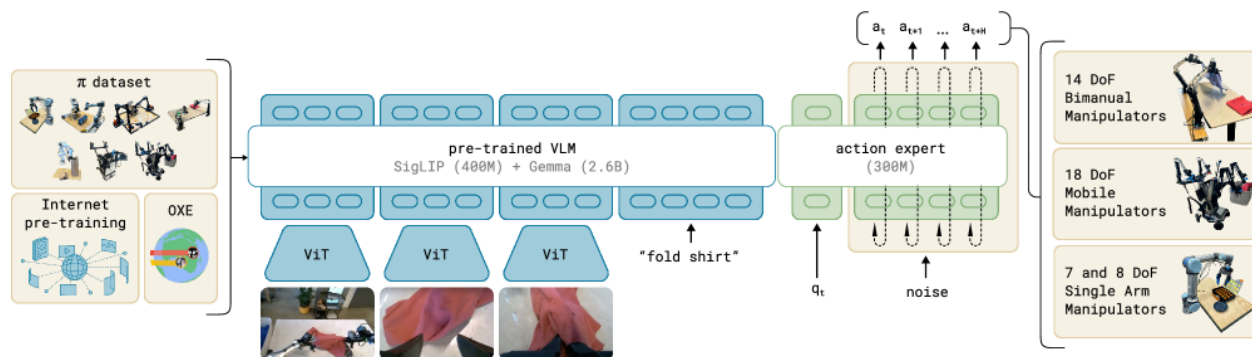


图 14: π_0 框架总览：视觉-语言骨干处理图像与语言指令，较小的动作专家以连续方式输出机器人动作

备注： π_0 内部用 flow matching 生成动作，这套机制留到讲 10 (Flow Matching) 和讲 11 (π_0 家族) 再拆。本讲只把它当一个”观测进、动作出”的黑盒——恰好印证第 2 节对策略的定义。

4.2 搭好舞台：LIBERO 仿真环境

LIBERO 是一个机器人操作的仿真基准，用程序化的方式生成了一批桌面操作任务，分成空间、物体、目标、长程四个套件。没有真机时，它就是我们验证策略的标准台子：

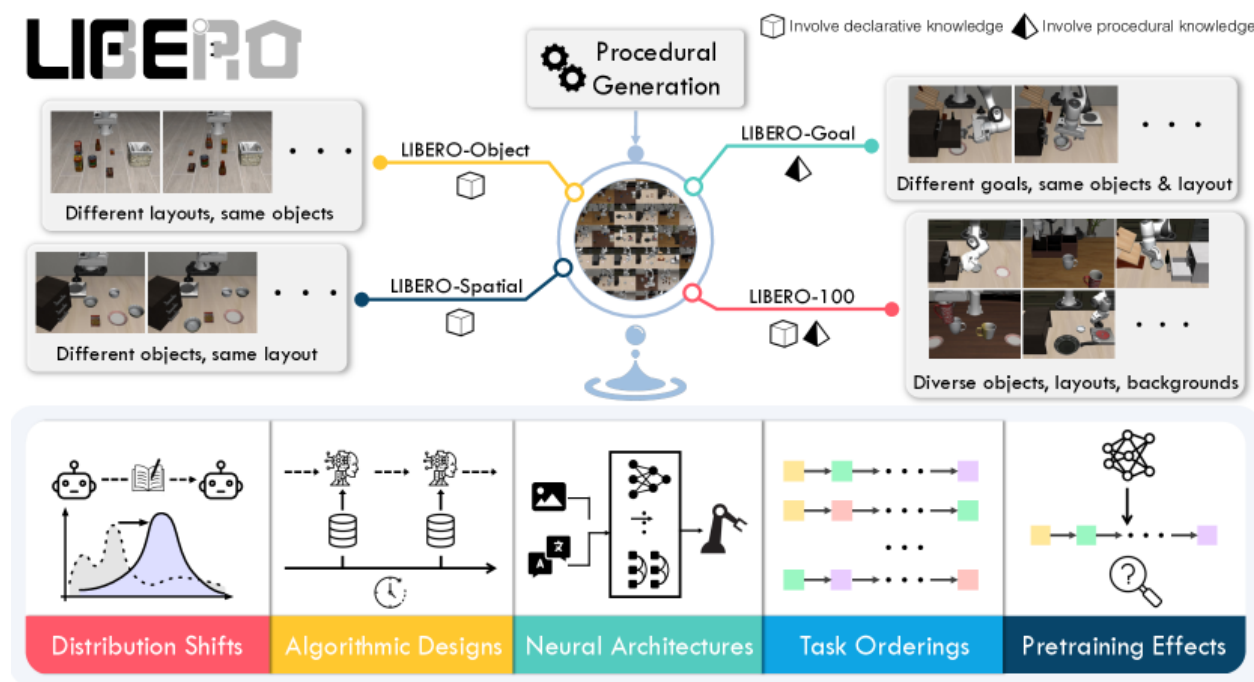


图 15: LIBERO 程序化生成的四个任务套件：空间、物体、目标与长程

4.3 闭环跑起来：看一眼、动一下

实验的核心是一个循环：从环境拿到当前观测（相机图 + 机器人状态 + 语言指令）→ 喂给 π_0 → 得到一段动作 → 在环境里执行 → 拿到新的观测……如此往复，直到任务完成或超时。这就是第 3 节反复强调的**闭环 rollout**：策略不是预测一次就完事，而是要在自己造成的后果里一直走下去。

4.4 拿着检查单核对这次部署

跑通只是第一步。真正把这一讲学透，是拿出 3.5 那张出发前检查单，把这次 π_0 + LIBERO 的运行逐行核对一遍：

- **动作口径**：LIBERO 约定的动作是 7 维的**相对增量**——前 6 维表示末端在位置和姿态上“再移动多少”，最后 1 维控制夹爪开合；环境在内部把增量累加成真实目标，正是 3.3 说的“相对口径要累加还原”。 π_0 的权重就是按这套口径训练的，两端说的是同一种话；一旦把它的输出错当绝对位姿发下去，立刻就是 3.3 右图的下场。

- **归一化统计量**: π_0 的 checkpoint 把归一化统计量和权重存放在一起, 加载策略时一起载入——推理时先把观测归一化送进网络, 再把输出反归一化成真实增量。“下载权重就能直接跑”, 靠的正是统计量没和权重分家 (3.4)。
- **数值量级**: 闭环跑起来之前, 可以先打印第一帧的动作看一眼: 7 维增量应当是贴近 0 的小数。如果看到大数, 那是口径或统计量出了问题的信号 (3.3、3.4 的症状), 先停下排查, 别急着跑完 rollout。
- **起始位姿**: LIBERO 每次重置会把机械臂和物体摆到任务定义的初始状态——正好落在训练分布之内, 3.2 那个雪球从源头上就不容易滚起来。
- **安全兜底**: 仿真里翻车零成本, 偶尔的离谱动作顶多让任务失败、摔不坏硬件, 这一关可以先放松; 但同样的闭环搬上真机之前, 裁剪、限速、限位必须补上 (3.5)。
- **评估标准**: 判断这次实验成不成, 看任务有没有在限定步数内完成, 不看 loss、也不看任何离线数字 (3.1、3.2)。

检查单六项, 每一项都能在这次实验里找到落点。把它核对完, 第 3 节那条”训练到部署的来回路”才算真正走通了一遍。

5 本讲小结

5.1 回顾整条数据流

这一讲只讲了一件事: **策略, 就是从观测到动作的那个网络; 用它, 就是让”观测 → 策略 → 动作 → 环境 → 新观测”这个闭环转起来。**训练时盯成功率而非只盯 loss, 部署时把动作口径、归一化统计量、安全边界对齐, 这个闭环才转得稳。

5.2 策略要吃数据: 下一讲采数据

策略这么强, 前提是有数据喂它。 π_0 的预训练权重背后, 是海量的示教数据。那这些数据从哪来、怎么采、采成什么格式才能直接训练? 下一讲就动手采一份能训练的数据集。

一句话总结: 端到端策略把”感知—规划—控制”的手写流水线, 压成一个”观测 → 动作”的网络; 学会它、跑通它的闭环、并在部署时对齐动作口径与归一化, 是后面所有 VLA 的起点。

参考资料

[1] CAPUANO F, PASCAL C, ZOUITINE A, et al. Robot learning: a tutorial[EB/OL]. (2025-10-14)[2026-06-30]. <https://arxiv.org/abs/2510.12403>.

- [2] BOJARSKI M, DEL TESTA D, DWORAKOWSKI D, et al. End to end learning for self-driving cars[EB/OL]. (2016-04-25)[2026-06-30]. <https://arxiv.org/abs/1604.07316>.
- [3] LEVINE S, FINN C, DARRELL T, et al. End-to-end training of deep visuomotor policies[EB/OL]. (2015-04-02)[2026-06-30]. <https://arxiv.org/abs/1504.00702>.
- [4] BROHAN A, BROWN N, CARBAJAL J, et al. RT-1: robotics transformer for real-world control at scale[EB/OL]. (2022-12-13)[2026-06-30]. <https://arxiv.org/abs/2212.06817>.
- [5] ZHONG Y, BAI F, CAI S, et al. A survey on vision-language-action models: an action tokenization perspective[EB/OL]. (2025-07-02)[2026-06-30]. <https://arxiv.org/abs/2507.01925>.
- [6] ROSS S, GORDON G J, BAGNELL J A. A reduction of imitation learning and structured prediction to no-regret online learning[EB/OL]. (2010-11-02)[2026-06-30]. <https://arxiv.org/abs/1011.0686>.
- [7] CHI C, XU Z, FENG S, et al. Diffusion policy: visuomotor policy learning via action diffusion[EB/OL]. (2023-03-07)[2026-06-30]. <https://arxiv.org/abs/2303.04137>.
- [8] BLACK K, BROWN N, DRIESS D, et al. π_0 : a vision-language-action flow model for general robot control[EB/OL]. (2024-10-31)[2026-06-30]. <https://arxiv.org/abs/2410.24164>.
- [9] LIU B, ZHU Y, GAO C, et al. LIBERO: benchmarking knowledge transfer for lifelong robot learning[EB/OL]. (2023-06-05)[2026-06-30]. <https://arxiv.org/abs/2306.03310>.